

Lezione 15 - Python e OOP - Base

COS'È OOP (PROGRAMMAZIONE ORIENTATA AGLI OGGETTI)

◆ Idea

👉 la **programmazione orientata agli oggetti (OOP)** è un modo di strutturare i programmi usando:

- **oggetti**

👉 ogni oggetto contiene:

- dati (*stato*)
- funzioni (*comportamento*)

◆ Esempio (lampadina)

👉 immaginiamo una lampadina:

- ha uno *stato* → accesa/spenta
- ha dei *comportamenti* → accendere, spegnere

👉 due lampadine:

- hanno lo stesso tipo di comportamento
- ma stato interno diverso

✳ sono due *istanze* dello stesso concetto

◆ Concetto di oggetto

👉 un oggetto rappresenta:

- un'entità del mondo reale (o astratta)

👉 è definito da:

- proprietà (dati)
- comportamenti (metodi)

◆ Struttura

👉 in OOP:

- raggruppiamo dati e funzioni insieme
- ogni oggetto gestisce il proprio stato

✳ codice più organizzato

◆ Vantaggi

👉 OOP permette:

- maggiore modularità
- codice più riutilizzabile
- migliore manutenibilità

◆ Idea importante

👉 in OOP:

- non pensiamo solo a funzioni
- ma a oggetti che fanno cose

✳ cambio di paradigma

CLASSI IN PYTHON

◆ Idea

👉 una **classe** è un “modello” (*blueprint*) per creare oggetti

👉 definisce:

- quali dati avrà un oggetto
- quali operazioni può fare

◆ Definizione di una classe

👉 in Python:

Codice:

```
class Cat:  
    pass
```

👉 `pass` significa:

- classe vuota
- esiste ma non ha ancora contenuto

◆ Classe vs oggetto

👉 la classe:

- è un modello astratto

👉 l'oggetto:

- è una istanza concreta della classe

◆ Creazione di un oggetto



Codice:

```
class Cat:

    def __init__(self):
        pass

tom = Cat()
print(tom)

silvestro = Cat()
print(silvestro)
```

◆ Spiegazione

👉 Cat() :

- crea una nuova istanza

👉 tom e silvestro :

- sono due oggetti diversi
- derivano dalla stessa classe

👉 anche se la classe è vuota:

- gli oggetti esistono comunque

◆ Idea importante

👉 classe:

- definisce il *tipo*

👉 oggetti:

- sono le *istanze*

✳️ stessa struttura, dati diversi

📌 COSTRUTTORE E ATTRIBUTI DI ISTANZA

◆ Idea

👉 quando creiamo un oggetto:

- vogliamo inizializzare i suoi dati

👉 per farlo usiamo il metodo:

✳️ `__init__`

◆ Il costruttore

👉 `__init__`:

- viene chiamato automaticamente alla creazione dell'oggetto
- serve per inizializzare gli attributi

◆ Sintassi

👉 il primo parametro è sempre:

`self`

👉 rappresenta:

- l'oggetto stesso

✳️ equivalente a `this` (in altri linguaggi)

◆ Esempio



Codice:

```
class Cat:

    def __init__(self, name, age):
        self.name = name
        self.age = age

tom = Cat("Tom", 13)
print(tom.name)
```

◆ Spiegazione

👉 quando scriviamo:

```
tom = Cat("Tom", 13)
```

👉 succede:

- viene creato l'oggetto
- viene chiamato `__init__`
- `name` e `age` vengono assegnati

◆ Attributi di istanza

👉 `self.name` e `self.age` :

- sono attributi dell'oggetto
- esistono per ogni istanza

👉 esempio:

- `tom.name = "Tom"`
- un altro oggetto può avere valori diversi

◆ Accesso agli attributi

👉 possiamo accedere così:

```
tom.name
```

👉 anche fuori dalla classe

◆ Idea importante

👉 gli attributi di istanza:

- sono specifici di ogni oggetto

✳️ ogni oggetto ha il proprio stato

📌 ATTRIBUTI DI CLASSE

◆ Idea

👉 alcuni dati sono **comuni a tutte le istanze**

👉 in questo caso:

- li definiamo a livello di classe

✳️ non dentro `__init__`

◆ Esempio



Codice:

```
class Cat:

    species = "Felis catus"

    def __init__(self, name, age):
        self.name = name
        self.age = age

tom = Cat("Tom", 13)
gerry = Cat("Gerry", 17)
```

◆ Spiegazione



species :

- è un attributo di classe
- condiviso da tutte le istanze

◆ Accesso agli attributi



Codice:

```
print(Cat.species)
print(tom.species)
print(gerry.species)
```



possiamo accedere:

- tramite la classe
- oppure tramite un oggetto

◆ Modifica tramite istanza (ATTENZIONE)



Codice:

```
tom.species = "gatto"
```

```
print(tom.species)  
print(gerry.species)
```

👉 succede:

- viene creato un attributo *locale* in `tom`
- NON viene modificata la classe

✳️ gli altri oggetti non cambiano

◆ Modifica tramite classe



Codice:

```
Cat.species = "gattino"
```

```
print(tom.species)  
print(gerry.species)
```

👉 succede:

- viene modificato l'attributo di classe
- tutte le istanze lo vedono aggiornato

◆ Idea importante

👉 attributi di classe:

- condivisi tra oggetti

👉 attributi di istanza:

- specifici di ogni oggetto

METODI DI ISTANZA

◆ Idea

👉 i **metodi di istanza** definiscono:

- il comportamento degli oggetti

👉 sono funzioni definite dentro la classe

◆ Esempio

Codice:

```
class Cat:

    species = "Felis catus"

    def __init__(self, name, age):
        self.name = name
        self.age = age

    def miagola(self):
        print(f"{self.name}: Miaooo!")

    def identificati(self):
        print(f"Sono {self.name} e sono un gatto di {self.age} anni")
```

◆ Creazione oggetti



Codice:

```
tom = Cat("Tom", 13)
gerry = Cat("Gerry", 17)
```

◆ Uso dei metodi



Codice:

```
tom.miagola()
tom.identificati()

gerry.miagola()
gerry.identificati()
```

◆ Spiegazione

👉 `miagola()` e `identificati()`:

- sono metodi della classe
- usano `self` per accedere agli attributi

👉 quando chiamiamo:

```
tom.miagola()
```

👉 Python passa automaticamente:

- `self = tom`

◆ Accesso ai dati

👉 dentro un metodo:

- accediamo agli attributi con `self.nome`

◆ Idea importante

👉 metodi di istanza:

- lavorano sui dati dell'oggetto

✳ ogni oggetto ha comportamento basato sul proprio stato

📌 PUBBLICO E PRIVATO IN PYTHON

◆ Problema

👉 in molti linguaggi esistono:

- `public`
- `protected`
- `private`

👉 Python invece usa:

✳ *convenzioni sui nomi*

📌 TIPI DI ACCESSO

◆ Pubblico

👉 nessuna convenzione speciale:

x = 5

👉 accessibile ovunque

◆ Protetto

👉 nome che inizia con `_`:

```
def _metodo(self):
```

👉 significa:

- uso interno
- ma accessibile comunque

✳ solo convenzione

◆ Privato

👉 nome che inizia con `__`:

```
def __metodo(self):
```

👉 Python applica:

✳ *name mangling*

📌 NAME MANGLING

◆ Idea

👉 il nome viene trasformato:

```
__data → _Queue__data
```

ESEMPIO (CLASSE)

Codice:

```
class Queue:

    def __init__(self):
        self.__data = []

    def enqueue(self, x):
        self.__data.append(x)

    def __do_stuff(self):
        self.__data = [1, 2, 3]
```

USO NORMALE

Codice:

```
q = Queue()
q.enqueue(3)
```

👉 accesso normale ai metodi pubblici

ACCESSO AI “PRIVATI”

Codice:

```
print(q._Queue__data)
q._Queue__do_stuff()
```

◆ Spiegazione

👉 `__data` e `__do_stuff`:

- vengono rinominati automaticamente
- NON sono realmente privati

👉 possiamo comunque accedere:

- usando il nome trasformato

◆ Perché usarlo?

👉 serve per:

- evitare accessi accidentali
- proteggere l'implementazione

✳️ non è sicurezza reale (*solo convenzione*)

◆ Idea importante

👉 in Python:

- il “privato” è solo una convenzione

✳️ responsabilità del programmatore

📌 METODI DI CLASSE

◆ Idea

👉 un **metodo di classe**:

- appartiene alla classe
- NON a una singola istanza

◆ Differenza

👉 metodi normali:

```
q = Queue()  
q.enqueue(3)
```

👉 metodi di classe:

```
Queue.metodo()
```

◆ A cosa servono?

👉 possiamo usarli per:

- creare oggetti in modi alternativi
- evitare funzioni globali

✳️ utili per costruzione di oggetti

📌 DEFINIZIONE

👉 si usa:

```
@classmethod
```

👉 il primo parametro è:

```
cls
```

👉 rappresenta:

- la classe (non l'oggetto)

ESEMPIO

Codice:

```
class Queue:

    def __init__(self):
        self.__data = []

    @classmethod
    def from_list(cls, lst):
        q = cls()
        q.__data = lst
        return q
```

UTILIZZO

Codice:

```
x = [1, 2, 5, 7, 4]
q = Queue.from_list(x)
```

◆ Spiegazione

👉 from_list :

- è un metodo della classe
- crea un oggetto a partire da una lista

👉 cls() :

- chiama il costruttore standard
- crea una nuova istanza

◆ Idea importante

👉 metodi di classe:

- lavorano sulla classe
- non su un oggetto specifico

✳️ utili per costruzione e utilità generali

📌 DUNDER METHODS (METODI SPECIALI)

◆ Idea

👉 i **dunder methods** (double underscore):

- sono metodi con nome speciale
- definiscono comportamenti standard degli oggetti

👉 esempio:

- `__init__`
- `__str__`

◆ A cosa servono?

👉 permettono di:

- usare operatori sugli oggetti
- accedere con indici
- stampare oggetti
- usare funzioni built-in

✳️ rendono gli oggetti “naturali” da usare

ESEMPIO BASE

◆ **init {#-init }**

👉 serve per:

- inizializzare un oggetto (*costruttore*)

Codice:

```
def __init__(self):  
    pass
```

oppure

Codice:

```
def __init__(self, ...):  
    self.x = ...
```

◆ **str {#-str }**

👉 serve per:

- rappresentazione in stringa

Codice:

```
def __str__(self):  
    return "testo"
```

👉 viene usato da:

```
print(obj)
```

CONFRONTO TRA OGGETTI

Metodo	Operazione
lt	<
le	<=
eq	==
ne	!=
gt	>
ge	>=

👉 tutti:

- prendono `self` e un altro oggetto
- restituiscono un booleano

ACCESSO CON CHIAVI

Metodo	Operazione
getitem	<code>obj[key]</code>
setitem	<code>obj[key] = val</code>
delitem	<code>del obj[key]</code>

👉 permettono:

- accesso tipo dizionario
- strutture personalizzate

OPERAZIONI MATEMATICHE

Metodo	Operazione
add	+
radd	+
iadd	+=
sub	-
mul	*

👉 permettono:

- definire operazioni tra oggetti

ALTRI METODI UTILI

Metodo	Operazione
len	len(obj)
contains	x in obj
call	obj(args)

◆ **call {#-call }**

👉 rende un oggetto:

✳️ “chiamabile” come una funzione



Codice:

```
f = Oggetto()  
f(x)
```

◆ Idea importante

👉 i dunder methods:

- collegano sintassi Python e oggetti

✳️ permettono di personalizzare il linguaggio



NUMERI DUALI (ESEMPIO AVANZATO)

◆ Idea

👉 un **numero duale** ha forma:

$$a + b\varepsilon$$

👉 permette di:

✳️ calcolare automaticamente le derivate

◆ Proprietà chiave

$$f(a + b\varepsilon) = f(a) + b f'(a) \varepsilon$$

✳️ contiene la derivata

STRUTTURA DELLA CLASSE

Codice:

```
class Dual:

    def __init__(self, a, b):
        self.a = a
        self.b = b
```

👉 rappresenta:

- $a \rightarrow$ valore
- $b \rightarrow$ coefficiente della derivata

SOMMA

Codice:

```
def __add__(self, other):
    if isinstance(other, Dual):
        return Dual(self.a + other.a, self.b + other.b)
    else:
        return Dual(self.a + other, self.b)
```

👉 gestisce:

- Dual + Dual
- Dual + numero

PRODOTTO

Codice:

```
def __mul__(self, other):  
    if isinstance(other, Dual):  
        return Dual(  
            self.a * other.a,  
            self.a * other.b + self.b * other.a  
        )  
    else:  
        return Dual(self.a * other, self.b * other)
```

👉 implementa:

$$(a + b\varepsilon)(c + d\varepsilon) = ac + (ad + bc)\varepsilon$$

SOTTRAZIONE

Codice:

```
def __sub__(self, other):  
    if isinstance(other, Dual):  
        return Dual(self.a - other.a, self.b - other.b)  
    else:  
        return Dual(self.a - other, self.b)
```


STAMPA

Codice:

```
def __str__(self):  
    return f"{self.a} + {self.b}ε"
```

UTILIZZO

Codice:

```
def f(x):  
    if (x.a > 3):  
        return x * 2  
    else:  
        return x * x
```

◆ Idea importante

👉 usando `Dual(a, 1)`:

- otteniamo sia valore che derivata

✳ base della differenziazione automatica (*derivate automatiche*)